

Cidades ESG Inteligentes

Documentação técnica de CI CD containerização e ambientes

Disciplina	Integrantes	Entrega
DevOps	Alvaro Miranda João Victor Vitor Viana Leonardo Sabbatini	Java Spring Boot Docker Compose GitHub Actions

Resumo

A solução é uma API REST para registrar e acompanhar iniciativas urbanas nos pilares Ambiental, Social e Governança. A entrega automatiza compilação, testes, construção e publicação da imagem, validação em staging e promoção da mesma imagem para produção.

Objetivo

Demonstrar um fluxo DevOps reproduzível, auditável e seguro, com falha rápida, ambientes isolados e evidências geradas automaticamente em cada execução.

Referências verificáveis

[Repositório do projeto](#)

[Execuções do GitHub Actions](#)

[Imagem no GitHub Container Registry](#)

1 Arquitetura da aplicação

A aplicação Spring Boot expõe endpoints REST, persiste os dados em PostgreSQL e publica sondas de saúde e identificação do ambiente pelo Spring Actuator. Staging e produção utilizam bancos, volumes, redes e perfis Spring independentes.

Componente	Staging	Produção
Aplicação	app-staging - porta 8081	app-production - porta 8082
Banco	db-staging	db-production
Volume	postgres-staging	postgres-production
Rede	staging-net	production-net
Perfil	staging	production

Endpoints entregues

Método	Rota	Finalidade
GET	/api/iniciativas	Listar iniciativas
POST	/api/iniciativas	Registrar iniciativa
PATCH	/api/iniciativas/{id}/status	Atualizar o status
GET	/actuator/health	Comprovar disponibilidade
GET	/actuator/info	Identificar o ambiente

2 Pipeline CI CD

O workflow é executado em pull requests, pushes na branch main e acionamentos manuais. Pull requests validam build, testes e imagem sem publicar nem implantar. Em main, a imagem é publicada no GHCR com a tag do commit e promovida sem rebuild.

Job	Dependência	Resultado
build-and-test	Código	Compilação, 4 testes e relatório Surefire
docker-image	Testes aprovados	Imagem única publicada no GHCR
deploy-staging	Imagem publicada	Compose, PostgreSQL e smoke test
deploy-production	Staging aprovado	Mesma imagem promovida e validada

Controles de qualidade

- Promoção interrompida automaticamente quando qualquer etapa falha.
- Produção não é executada em pull requests.
- A senha pode ser fornecida pelo secret POSTGRES_PASSWORD e não é versionada.
- Os ambientes usam GitHub Environments e podem exigir aprovação manual.
- Logs, estado dos containers e resultados dos smoke tests são publicados como artefatos.

3 Containerização e orquestração

O Dockerfile utiliza build multi-stage. Maven e JDK 21 existem somente na etapa de compilação; a imagem final utiliza JRE 21 Alpine e executa com usuário não-root. O arquivo .dockerignore reduz o contexto e evita o envio de arquivos locais e documentação para a imagem.

Estratégia	Implementação
Imagem enxuta	JRE Alpine na etapa final
Segurança	Usuário spring sem privilégios
Disponibilidade	HEALTHCHECK em /actuator/health
Persistência	Volumes PostgreSQL independentes
Isolamento	Redes e profiles distintos por ambiente
Configuração	Variáveis de ambiente e .env.example

Comandos principais

```
docker compose --env-file .env --profile staging up --build -d
```

```
docker compose --env-file .env --profile production up --build -d
```

```
docker compose --profile staging --profile production down -v
```

4 Testes e validação automatizada

O Maven executa quatro testes de integração com Spring Boot, MockMvc e H2: criação e listagem de iniciativa, retorno 404 para atualização inexistente, health e exposição do endpoint info do Actuator. O smoke test de cada deploy valida saúde, identificação do ambiente, escrita e leitura no PostgreSQL.

Camada	Verificações
JUnit e MockMvc	POST, GET, PATCH inexistente e health
Smoke test	health, info, POST e GET
Banco	Persistência real em PostgreSQL no Compose
Evidência	Summary, relatórios, logs e artefatos por SHA

5 Desafios e soluções

Desafio	Solução adotada
Separar ambientes	Profiles, redes, volumes, bancos e perfis Spring distintos
Promover o mesmo artefato	Imagem tagueada pelo SHA e reutilizada nos dois deploys
Evitar deploy em PR	Condição por evento nos jobs de staging e produção
Comprovar o funcionamento	Smoke test e artefatos de evidência gerados pelo pipeline
Proteger credenciais	Secret opcional e arquivos locais ignorados

6 Evidências da execução

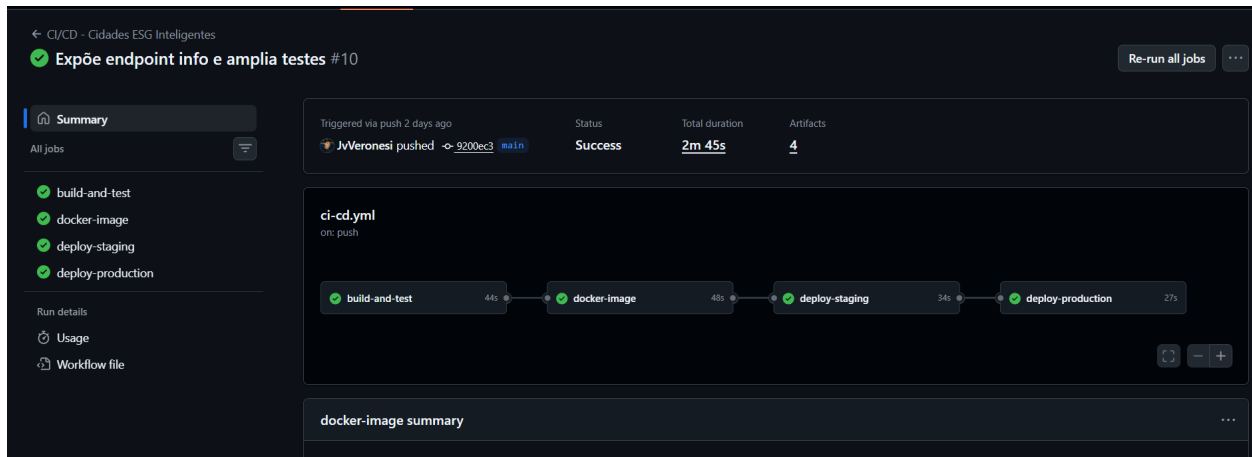


Figura 1 - Pipeline completo com os quatro jobs aprovados.

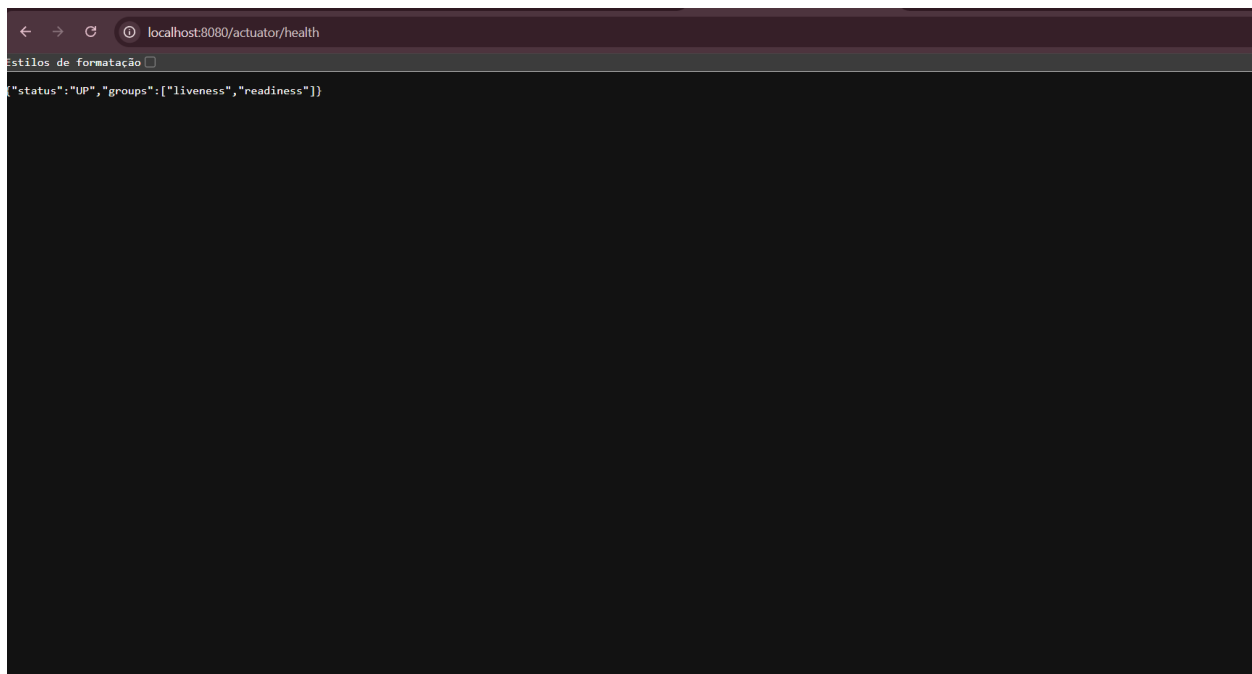


Figura 8 - Aplicação local saudável, com status UP no Spring Actuator.

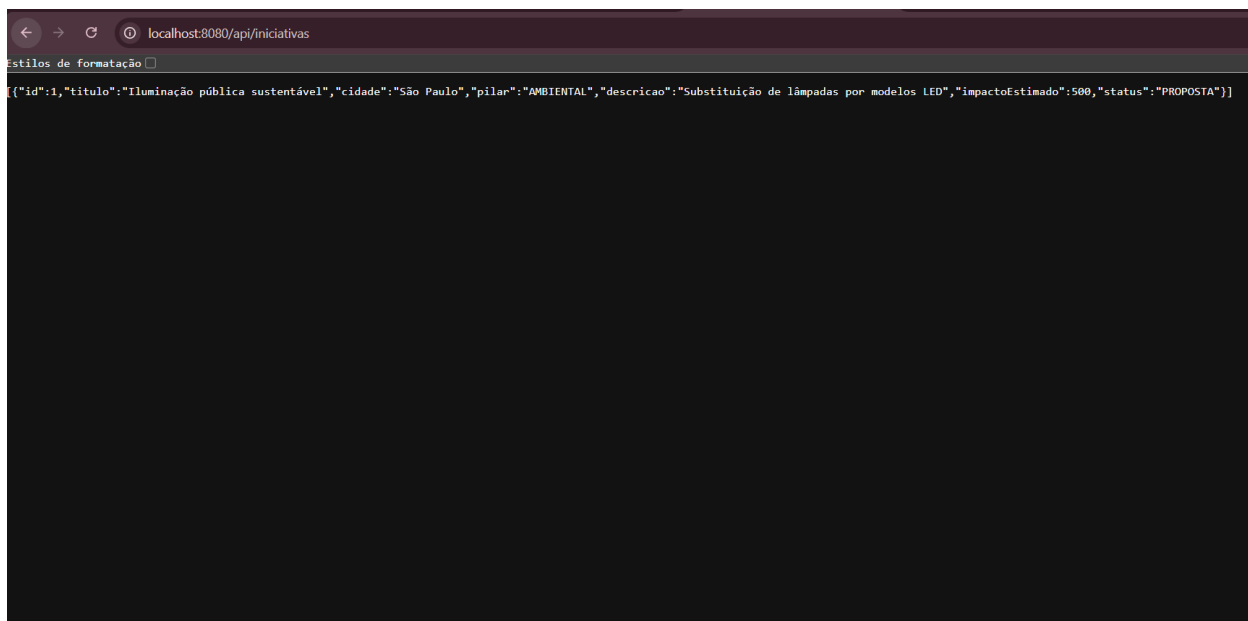


Figura 9 - API local listando a iniciativa ESG cadastrada.

7 Checklist final

Item	Status
Projeto compactado e organizado	OK
Dockerfile multi-stage funcional	OK
Docker Compose com volumes, redes e variáveis	OK
Pipeline com build, testes, imagem e dois deploys	OK
README com execução, arquitetura e evidências	OK
Documentação técnica em PDF	OK
Evidências automáticas por execução	OK
Capturas obrigatórias do CI/CD	1/7
Evidências locais complementares	2/2

A rastreabilidade é garantida pelo SHA usado simultaneamente na execução do workflow, na tag da imagem e nos nomes dos artefatos de evidência.